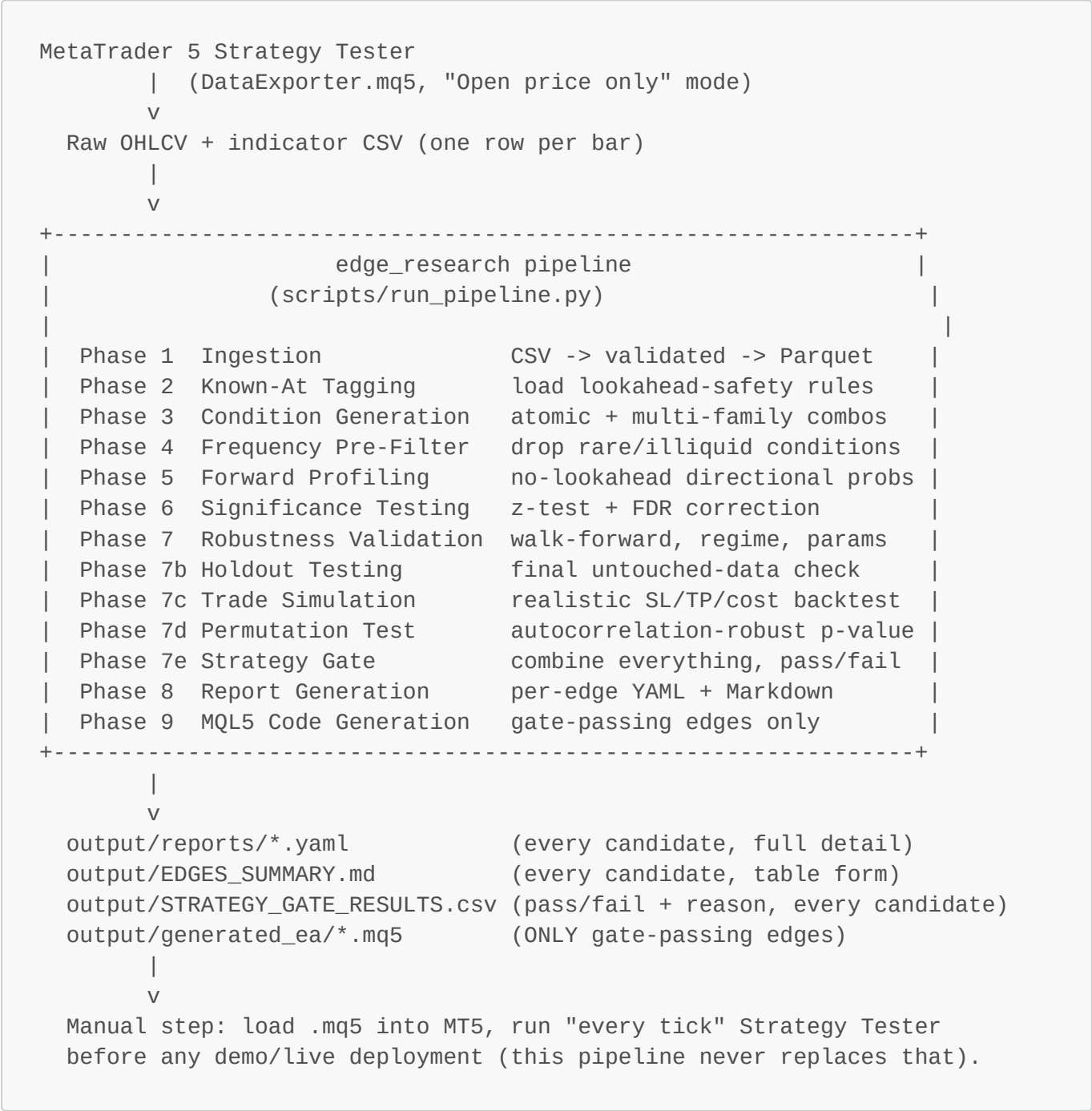


How This Project Works

This document explains the mechanics of the `edge_research` pipeline end to end: what each phase does, what files are involved, how data flows through the system, how to run it, and how to read its output. For *why* the project is built this way (especially the Phase 7c–7e pivot toward strategy construction), see `PROJECT_DIRECTION.md`. This document is the "how," that one is the "why."

1. The big picture



Everything upstream of Phase 7c answers "does this condition predict direction." Phase 7c onward answers "would trading this condition, with real costs and a real exit scheme, actually make money" — see `PROJECT_DIRECTION.md` for why that distinction matters.

2. Where the data comes from (outside this repo)

`DataExporter.mq5` runs inside the MetaTrader 5 Strategy Tester in "Open price only" mode. For every completed bar it writes one CSV row containing OHLCV plus ~80 precomputed indicator values (moving averages, RSI, CCI, Stochastic, MACD, Bollinger Bands, ATR, volume-based indicators, etc.). Using MT5 itself to compute indicators — rather than recomputing them in Python — eliminates any risk of formula mismatch between what was "discovered" and what the eventual EA will actually see live.

The exported CSV is placed under `data/raw_csv/` and is the single input to Phase 1.

3. Phase-by-phase mechanics

Phase 1 — Ingestion & Storage (`edge_research/ingestion/loader.py`)

Reads the raw CSV, parses the `time` column into a proper datetime index, converts all indicator columns to `float32` (memory efficiency — the M1 dataset is ~5.5M rows x ~90 columns), and writes the result to a Parquet file under `output/storage/` for fast reloading by every later phase. Duplicate or non-monotonic timestamps are detected and handled here — this stage exists specifically because an earlier version of the MQL5 exporter had a bug that produced thousands of duplicate-timestamp rows, which silently corrupted every downstream statistic until it was caught.

Phase 2 — Known-At Tagging (`edge_research/known_at/tagging.py`)

Loads `config/known_at_delay.yaml`, a set of rules describing which indicator values are "known" at the close of a given bar versus only available with some reporting delay. This exists to keep every later lookahead-free evaluation honest.

Phase 3 — Condition Generation

(`edge_research/conditions/condition_library.py`)

Two kinds of candidate trading conditions are generated:

- **Atomic conditions:** a single indicator compared to a threshold, e.g. `rsi_14 < 20.0`. Thresholds come from `pipeline_config.yaml` (`rsi_thresholds`, `cci_thresholds`, `mom_thresholds`, `stoch_thresholds`), plus MA-vs-close conditions if `include_ma_conditions: true`.
- **Combined conditions:** an AND of 2 to `max_atoms_per_combo` atomic conditions, e.g. `rsi_21 < 20.0 AND cci_20 < -100.0 AND stoch_14_3_3_k < 20.0`.

Combinations are NOT built from all atomic conditions (that would be combinatorially explosive and statistically unsound — see `PROJECT_DIRECTION.md` section 2b on multiple-testing risk). Instead:

1. All atomic conditions are cheaply ranked by their 1-bar-ahead effect size.
2. The ranking is **stratified by indicator family** (`rsi`, `cci`, `stoch`, `mom`, `ma`) so no single family (historically RSI, which tends to have the largest raw effect sizes) monopolizes the pool — `top_n_atomic_for_combine` conditions are pulled roughly evenly across families.
3. Combinations are only formed between atoms from **different** families (`is_diverse_combo` in `scripts/run_pipeline.py`), both via a correlation check (`max_corr_combine`) and a hard family-

name check — so `rsi_14 < 20 AND rsi_21 < 25` is never generated even if their measured correlation happens to be below the cutoff for some dataset.

Phase 4 — Frequency Pre-Filter (`edge_research/conditions/frequency_filter.py`)

Drops any condition that doesn't trigger often enough to be statistically trustworthy or practically tradeable: `min_occurrence` (absolute count) and `min_freq_per_year` (rate) both must be satisfied.

Phase 5 — Forward Profiling (`edge_research/forward_profile/engine.py`)

For every bar and every horizon from 1 to `max_horizon`, precomputes whether `close[entry+horizon] > open[entry]` (a "bull" outcome), where `entry = trigger_bar + 1` — i.e. the decision is made at the close of the trigger bar, but the trade is only assumed to enter at the *next* bar's open, so there is no lookahead. This full precomputed matrix (`forward_matrix`, shape `n_bars x max_horizon`) is what every significance test and the Phase 7d permutation test read from.

Phase 6 — Significance Testing (`edge_research/forward_profile/significance.py`)

For each condition, runs a two-sided proportion z-test at every horizon against the unconditional baseline probability, then applies Benjamini-Hochberg FDR correction — first within each condition (across horizons), then globally across all conditions and horizons pooled together. The horizon with the largest significant effect size becomes that condition's "optimal horizon," used by every phase after this one.

Phase 7 — Robustness Validation (`edge_research/validation/`)

Four independent checks, all pre-existing from before the strategy- construction pivot:

- **Walk-forward** (`walk_forward.py`): re-evaluates the condition across rolling train/test windows (`wf_train_bars/wf_test_bars/wf_step_bars`) and flags whether its effect direction is consistent (`edge_direction_consistent`) across windows, not just in the full sample.
- **Regime stress test** (`regime_stress_test.py`): checks performance separately in trending vs. ranging and high- vs. low-volatility regimes (classified via ADX and ATR percentiles).
- **Parameter sensitivity** (`parameter_sensitivity.py`): shifts the condition's numeric threshold by $\pm \text{param_shift_pct}$ and checks the edge doesn't collapse — only meaningful for conditions with a genuine numeric threshold (skipped for MA-vs-close and combined conditions; see the comments in `scripts/run_pipeline.py`).
- **Cost model** (`cost_model.py`): a simpler, older cost estimate than Phase 7c's full simulation; kept for backward compatibility and reporting.

This phase is currently the slowest part of the pipeline on the full M1 dataset (walk-forward alone re-evaluates every condition across every window). If pipeline iteration speed becomes a bottleneck, this is the first place to profile.

Phase 7b — Holdout Set Testing

The last `holdout_pct` fraction of the dataset (chronologically) is never touched by any earlier phase's threshold tuning. Each surviving condition's probability is recomputed on just this untouched slice as a final, independent sanity check.

Phase 7c — Realistic Trade Simulation

([edge_research/simulation/trade_simulator.py](#))

Added in the strategy-construction pivot. Bar-by-bar simulation of actually trading the condition: entry at next bar's open (plus half the spread+slippage cost), an ATR-sized stop loss and take profit (optionally a trailing stop), a maximum holding period, and exit cost on the way out. Produces win rate, expectancy in price units and in R-multiples, profit factor, and a max-drawdown figure. See [PROJECT_DIRECTION.md](#) sections 4 and 6 for the full rationale and definitions, and section 2 of this document's troubleshooting notes below for the spread/pip_value unit-mismatch pitfall that can silently make this phase unrealistically optimistic.

Trade direction ([long/short](#)) is inferred automatically per condition by comparing its optimal-horizon probability to the unconditional baseline ([infer_direction](#) in [trade_simulator.py](#)).

Phase 7d — Rotation-Based Permutation Test

([edge_research/validation/permutation_test.py](#))

Added in the pivot. An autocorrelation-robust significance check that complements the Phase 6 z-test (which assumes independent samples — a poor assumption for autocorrelated bar-to-bar market data). Builds a null distribution by circularly rotating the real forward-outcome sequence by many random offsets, and computes an empirical p-value from how often a random rotation produces an effect size at least as large as the real, unrotated one. See [PROJECT_DIRECTION.md](#) section 4 for the full rationale.

Phase 7e — Final Strategy Selection Gate

([edge_research/selection/strategy_gate.py](#))

Combines every check above into one pass/fail decision. **All** enabled gates must pass:

1. Phase 6 significance ([p_adj](#) < [alpha](#))
2. Walk-forward direction consistency
3. Phase 7d permutation significance
4. Phase 7c has at least [gate_min_sim_trades](#) simulated trades
5. Phase 7c [expectancy_r](#) > 0
6. Phase 7c [profit_factor](#) >= [gate_min_profit_factor](#)

Every condition's pass/fail result and the specific reasons for failure are written to [output/STRATEGY_GATE_RESULTS.csv](#) — this is the single best file to open first when trying to understand a given run's results.

Phase 8 — Report Generation ([edge_research/reporting/](#))

Writes one YAML file per candidate condition ([output/reports/*.yaml](#), every candidate, not just gate-passers — kept for transparency and future reference) and one combined Markdown summary ([output/EDGES_SUMMARY.md](#)).

Phase 9 — MQL5 Code Generation ([edge_research/mql5_codegen/generator.py](#))

Generates one standalone [.mq5](#) Expert Advisor file per condition that passed the Phase 7e gate — and *only* those. This is the pivot's central behavior change: earlier versions of this pipeline generated an EA for every statistically significant condition regardless of realistic tradeability; now generation is gated on realistic

profitability. Following this project's established MQL5 conventions: bar-gated processing, signals evaluated on the last completed bar, ATR-based safety stops, virtual position tracking, CSV trade logging, unique magic numbers, and every EA as a fully separate file (never combined).

4. Configuration reference (`config/pipeline_config.yaml`)

Configs are organized per-timeframe under `config/<TF>/pipeline_config.yaml` (e.g. `config/M1/`, `config/D1/`) because walk-forward window sizes and frequency thresholds need very different values depending on how many bars a timeframe produces. Pass `--config-dir config/<TF>` to `run_pipeline.py` to select one; passing just `--config-dir config` uses whatever top-level defaults are checked into that file, which may not be tuned for the specific run you're about to do — always double check.

Key groups:

Group	Keys	Purpose
Horizon	<code>max_horizon</code> , <code>min_horizon</code>	forward-looking window range (Phase 5)
Frequency filter	<code>min_occurrence</code> , <code>min_freq_per_year</code>	Phase 4 thresholds
Significance	<code>alpha</code> , <code>fdr_method</code>	Phase 6 test parameters
Walk-forward	<code>wf_trainBars</code> , <code>wf_testBars</code> , <code>wf_stepBars</code>	Phase 7 window sizing — must fit the dataset's actual bar count, see the in-file scaling guidance
Regime	<code>atr_percentile_low/high</code> , <code>adx_trending/ranging_threshold</code>	Phase 7 regime classification
Sensitivity	<code>param_shift_pct</code>	Phase 7 parameter shift size
Cost model	<code>spread_pips</code> , <code>slippage_pips</code> , <code>pip_value</code>	symbol-dependent — see troubleshooting below
Holdout	<code>holdout_pct</code>	Phase 7b split fraction
Trade sim	<code>sim_sl_atr_mult</code> , <code>sim_tp_atr_mult</code> , <code>sim_use_trailing_stop</code> , <code>sim_trail_atr_mult</code> , <code>sim_atr_column</code> , <code>sim_max_holding_bars</code>	Phase 7c exit scheme
Permutation	<code>permutation_n</code> , <code>permutation_random_state</code>	Phase 7d sample count / seed
Gate	<code>gate_min_sim_trades</code> , <code>gate_min_profit_factor</code> , <code>gate_require_walk_forward</code> , <code>gate_require_permutation</code>	Phase 7e thresholds

Group	Keys	Purpose
Condition generation	<code>rsi_thresholds</code> , <code>cci_thresholds</code> , <code>mom_thresholds</code> , <code>stoch_thresholds</code> , <code>include_ma_conditions</code>	Phase 3 atomic condition space
Combination	<code>top_n_atomic_for_combine</code> , <code>max_corr_combine</code> , <code>max_atoms_per_combo</code>	Phase 3 combined condition space

Critical pitfall: `spread_pips` / `pip_value` are forex-shaped, not symbol-agnostic

`spread_price_units = spread_pips * pip_value`. The checked-in defaults (`spread_pips: 2.0`, `pip_value: 0.0001`) are correct for 5-digit forex majors. For XAUUSD (or any non-forex instrument), these MUST be overridden so the product matches that symbol's real spread in price units (established earlier in this project: XAUUSD spread $\approx$ 0.3 price units — so e.g. `spread_pips: 30.0`, `pip_value: 0.01`). Getting this wrong does not crash anything — it silently makes Phase 7c's simulated `expectancy_r` and the Phase 7e gate unrealistically optimistic (cost too low) or unrealistically pessimistic (cost too high).

Always check this override before trusting a run's `STRATEGY_GATE_RESULTS.csv`.

5. Output files reference

File	Written by	Contents
<code>output/storage/<SYMBOL>_<TF>.parquet</code>	Phase 1	validated, typed OHLCV+indicator data
<code>output/reports/<edge_id>.yaml</code>	Phase 8	full detail for one candidate (significance, robustness, holdout, gate status)
<code>output/EDGES_SUMMARY.md</code>	Phase 8	table of every candidate
<code>output/STRATEGY_GATE_RESULTS.csv</code>	Phase 7e	every candidate's pass/fail + specific failure reasons + key sim/permutation numbers — start here when reviewing a run
<code>output/generated_ea/<edge_id>.mq5</code>	Phase 9	one EA per gate-passing condition only

6. How to run

```
python scripts/run_pipeline.py \
  data/raw_csv/<SYMBOL>_<TF>_export_<DATE>.csv \
  --symbol <SYMBOL> \
  --timeframe <TF> \
  --config-dir config/<TF> \
  --output-dir output
```

Recommended workflow: always smoke-test on D1 first (a few thousand bars, finishes in well under a minute) before committing to a full M1 run (millions of bars, can take hours — see Phase 7's note on runtime above). A config or code change that looks right on paper should be verified on D1 before spending hours re-running M1.

7. Troubleshooting checklist

"0 edges passed the gate" — Not necessarily a bug. Open `STRATEGY_GATE_RESULTS.csv` and tally the `reasons` column. If most failures are cost/expectancy related, check the `spread/pip_value` override above first; if that's already correct, it may genuinely mean this timeframe's discovered edges are too small to survive real trading costs — a legitimate research conclusion, not a pipeline failure.

"No valid windows for walk-forward validation" spam — ``wf_train_bars`

- `wf_test_bars` doesn't fit inside the dataset's actual bar count for this timeframe. Rescale per the guidance comment at the top of `pipeline_config.yaml`, or use the correct per-timeframe config directory.

A run takes far longer than expected — Phase 7 (walk-forward + regime + parameter sensitivity, pre-existing code) is currently the dominant cost on large datasets. Reducing the number of walk-forward windows (larger `wf_step_bars`) or the size of the combined-condition pool (`top_n_atomic_for_combine`, `max_atoms_per_combo`) are the fastest levers to pull.

Combined conditions all look like variations of one indicator family — check that `include_ma_conditions` and enough distinct families are actually present in the log line `"Selected top N atomic conditions for combination (stratified across K families: [...])"` at Phase 3. If `max_atoms_per_combo` exceeds the number of available families, larger combos are mathematically impossible and will silently generate zero results at that size (a warning is logged when this happens).

8. Glossary

- **Atomic condition:** a single indicator-vs-threshold comparison.
- **Combined condition:** an AND of multiple atomic conditions from different indicator families.
- **Optimal horizon:** the forward-looking bar count (1 to `max_horizon`) at which a condition's effect size is largest among its FDR-significant horizons.
- **Effect size:** the difference between a condition's forward "bull" probability and the unconditional baseline probability, at its optimal horizon.
- **R-multiple:** trade P&L divided by the initial stop-loss distance; the standard way to compare trades with different stop sizes.
- **Expectancy:** average P&L per trade (in price units or R); the number that actually determines long-run profitability, not win rate.
- **Profit factor:** gross winning P&L divided by gross losing P&L across simulated trades; 1.0 is exact breakeven before unmodeled costs.
- **Gate:** the Phase 7e pass/fail decision combining every independent validation check; only gate-passing conditions reach MQL5 generation.